jQuery & Ajax

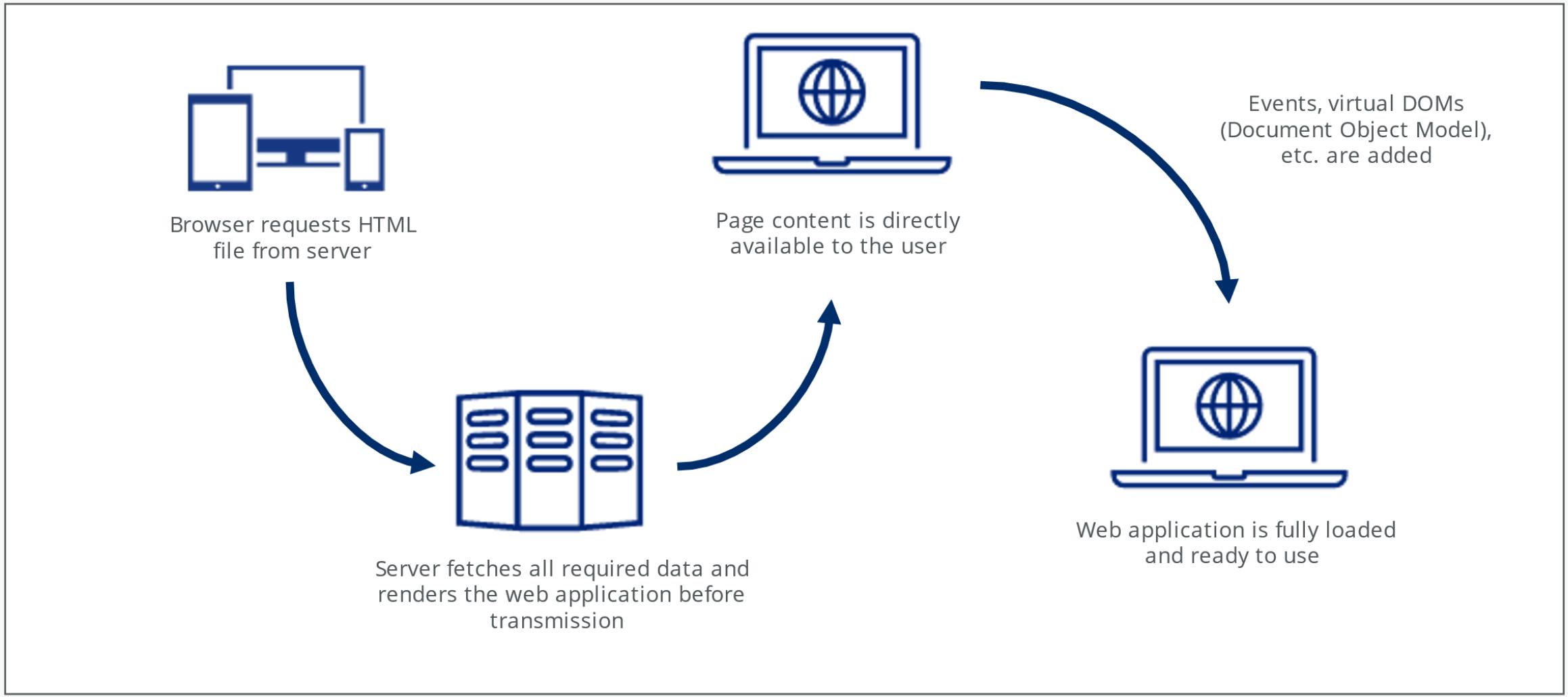
Att bygga interaktiva webbplatser

Dagens föreläsing

- jQuery, varför?
- jQuery, syntax
- Lär känna din HTML-kod
- DOM – Nyckeln till interaktiva webbplatser
- Case, förstå sin HTML-kod
  - Metoder för att lösa problem
- Ajax, vad är det?
- Ajax, syntax
- Case - Ajax

Framworks & Libraries

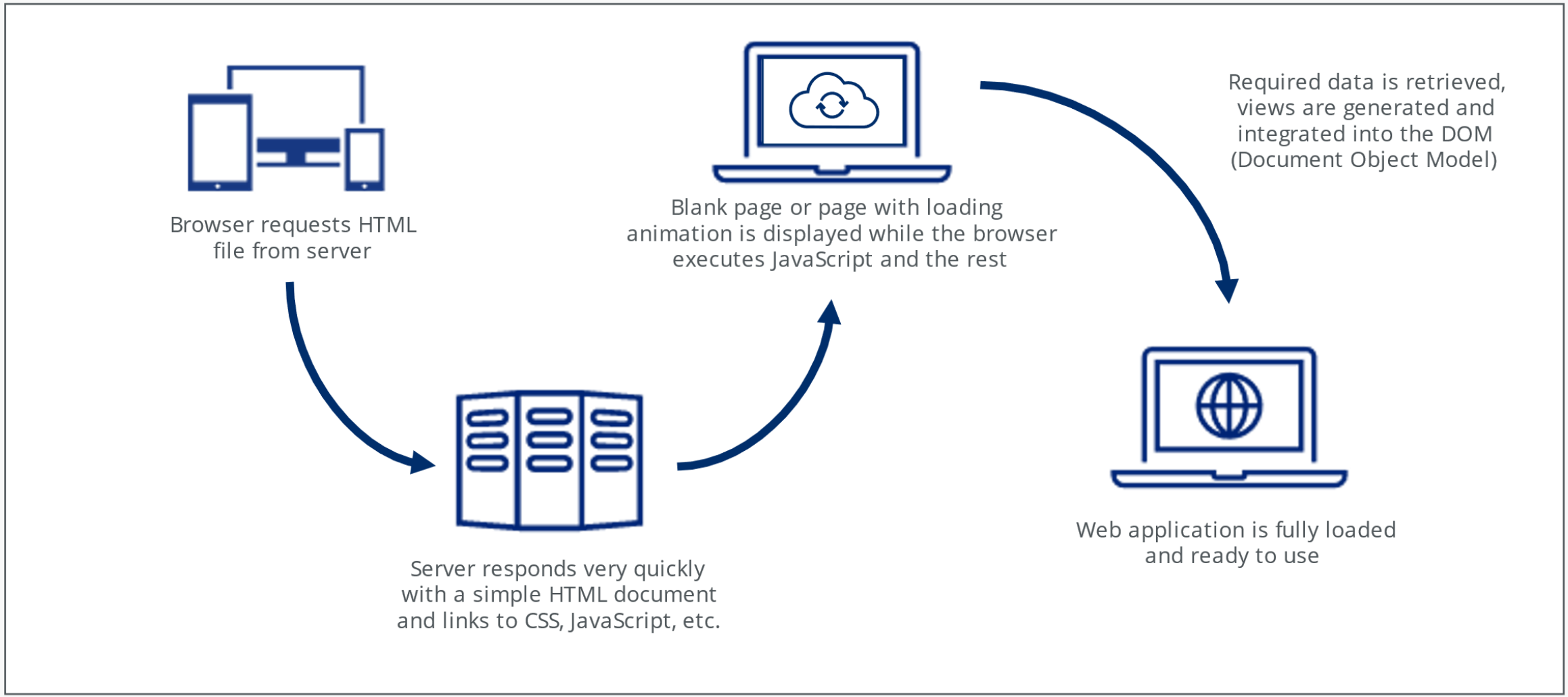
Server side rendering



IONOS

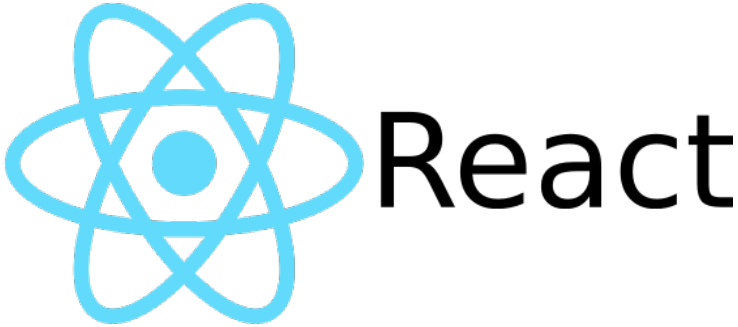
Bibliotek

Client side rendering



IONOS

Ramverk



Not An Imposter: Fighting Front-End Fatigue

By David Berner

November 17th, 2016 | Techniques, Workflow | 63 Comments

I recently spoke with a back-end developer friend about **how many hours I spend coding or learning about code outside of work**. He showed me a passage from an Uncle Bob book, "Clean Code", which compares the hours musicians spend with their instruments in preparation for a concert to developers rehearsing code to perform at work.

Advertisement

I like the analogy but I'm not sure I fully subscribe to it; it's that type of thinking that can cause burnout in the first place. I think it's great if you want to further your craft and broaden your skill set, but to be doing it every hour of the day isn't sustainable.

Front-end fatigue is very real. I've seen a number of posts on JavaScript fatigue but I think the problem extends further than that specific language.

To be clear, this isn't another rant about how it's all bad and everything is moving too fast — I love that technology is evolving so rapidly. Equally, I can appreciate how it can be

https://www.smashingmagazine.com/2016/11/not-an-imposter-fighting-front-end-fatigue/

Search on Smashing Maga
e.g. JavaScript

Advertisement

PSD2HTML.com

Smashing

Subscribe to our email nev
and valuable resources, se
Tuesday.
email address
234,898 Subscribers
powered by MailChi

Advertisement

Smashing Jobs

JOB ADVERTS EXPLAINED

WHAT IT SAYS	WHAT IT MEANS
JOB TITLE: Senior Full-stack Agile architect + ninja dev	JOB TITLE: Web developer
JOB DESCRIPTION: We're looking for a maker. Someone to shake up the industry and innovate in the digital space.	JOB DESCRIPTION: We're looking for someone semi-normal to get us out of the mess we're in.
REQUIREMENTS: JS (ES6, ES7), React, Flux, Angular 1/2, nodeJS, SASS, OOCSS, BEM, PostCSS, Agile, PHP, .NET, C#, Ruby, SVG, SEO, canvas, WebGl, animation, photoshop, WebAssembly, 3D, VR, AR ++	REQUIREMENTS fundamental knowledge. Ability to learn.

jQuery

Categories

- Events
- Foundation
- jQuery
- jQuery UI
- Projects
- Weekly News

Recent Posts

- jQuery 4.0.0
- jQuery 4.0.0 Release Candidate 1
- Second Beta of jQuery 4.0.0
- Upgrading jQuery: Working Towards a Healthy Web
- jQuery 4.0.0 BETA!
- jQuery 3.7.1 Released: Reliable Table Row Dimensions
- jQuery 3.7.0 Released: Staying in Order
- jQuery 3.6.4 Released: Selector Forgiveness
- jQuery 3.6.3 Released: A Quick Selector Fix

jQuery 4.0.0

Posted on January 17, 2026 by Timmy Willison

On January 14, 2006, John Resig introduced a JavaScript library called jQuery at BarCamp in New York City. Now, 20 years later, the jQuery team is happy to announce the final release of jQuery 4.0.0. After a long development cycle and several pre-releases, jQuery 4.0.0 brings many improvements and modernizations. It is the first major version release in almost 10 years and includes some breaking changes, so be sure to read through the details below before upgrading. Still, we expect that most users will be able to upgrade with minimal changes to their code.

Many of the breaking changes are ones the team has wanted to make for years, but couldn't in a patch or minor release. We've trimmed legacy code, removed some previously-deprecated APIs, removed some internal-only parameters to public functions that were never documented, and dropped support for some "magic" behaviors that were overly complicated.

We have an upgrade guide and jQuery Migrate plugin release ready to assist with the transition. Please upgrade and let us know if you encounter any issues.

As usual, the release is available on our CDN and the npm package manager. Other third party CDNs will probably have it available soon as well, but remember that we don't control their release schedules and they will need some time. Here are the highlights for jQuery 4.0.0.

IE<11 support removed

jQuery 4.0 drops support for IE 10 and older. Some may be asking why we didn't remove support for IE 11. We plan to remove support in stages, and the next step will be released in jQuery 5.0. For now, we'll start by removing code specifically supporting IE versions older than 11.

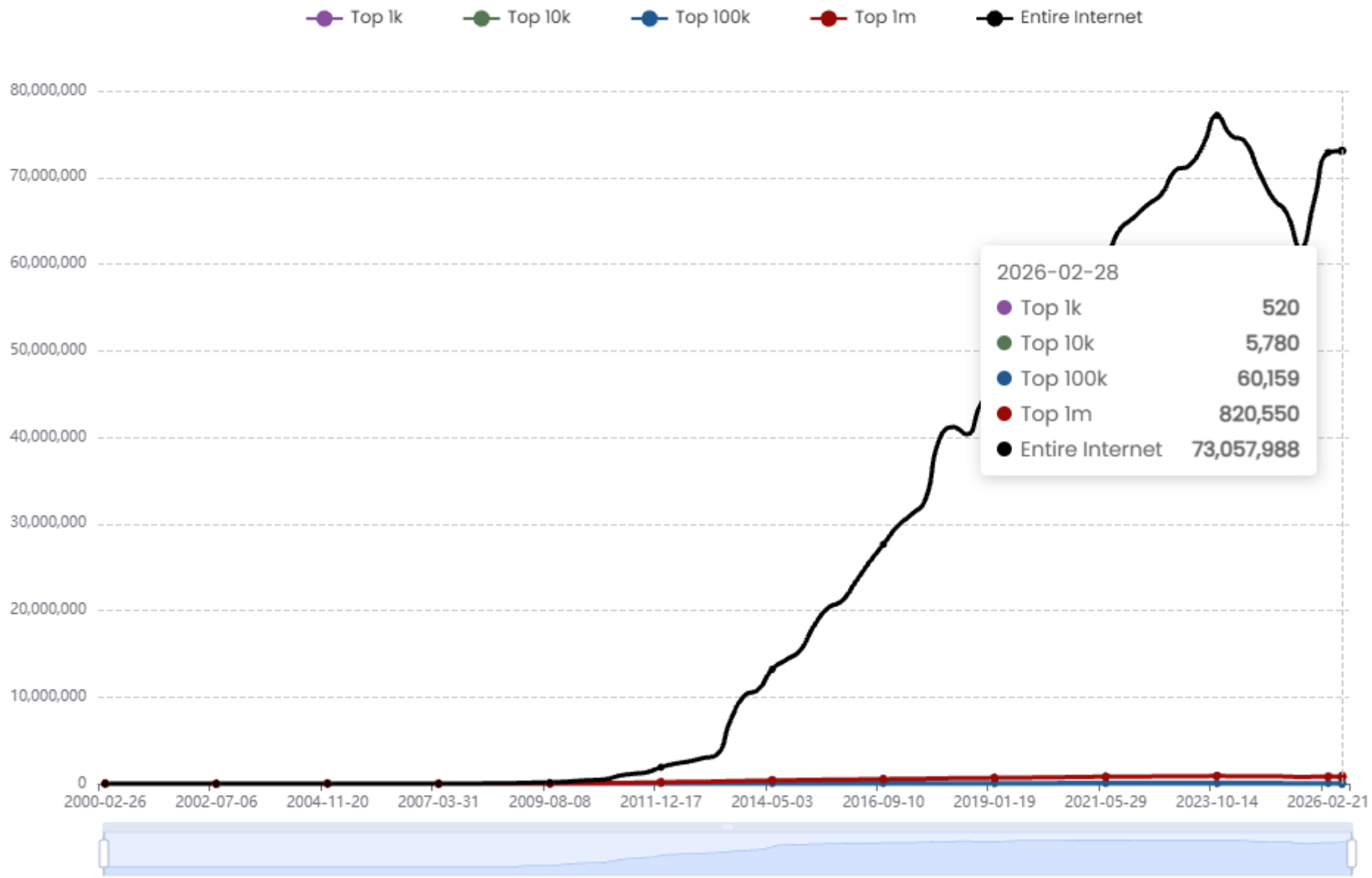
We also dropped support for other very old browsers, including Edge Legacy, iOS versions earlier than the last 3, Firefox versions earlier than the last 2 (aside from Firefox ESR), and Android Browser. No changes should be required on your end. If you need to support any of these browsers, stick with jQuery 3.x.

Trusted Types and CSP

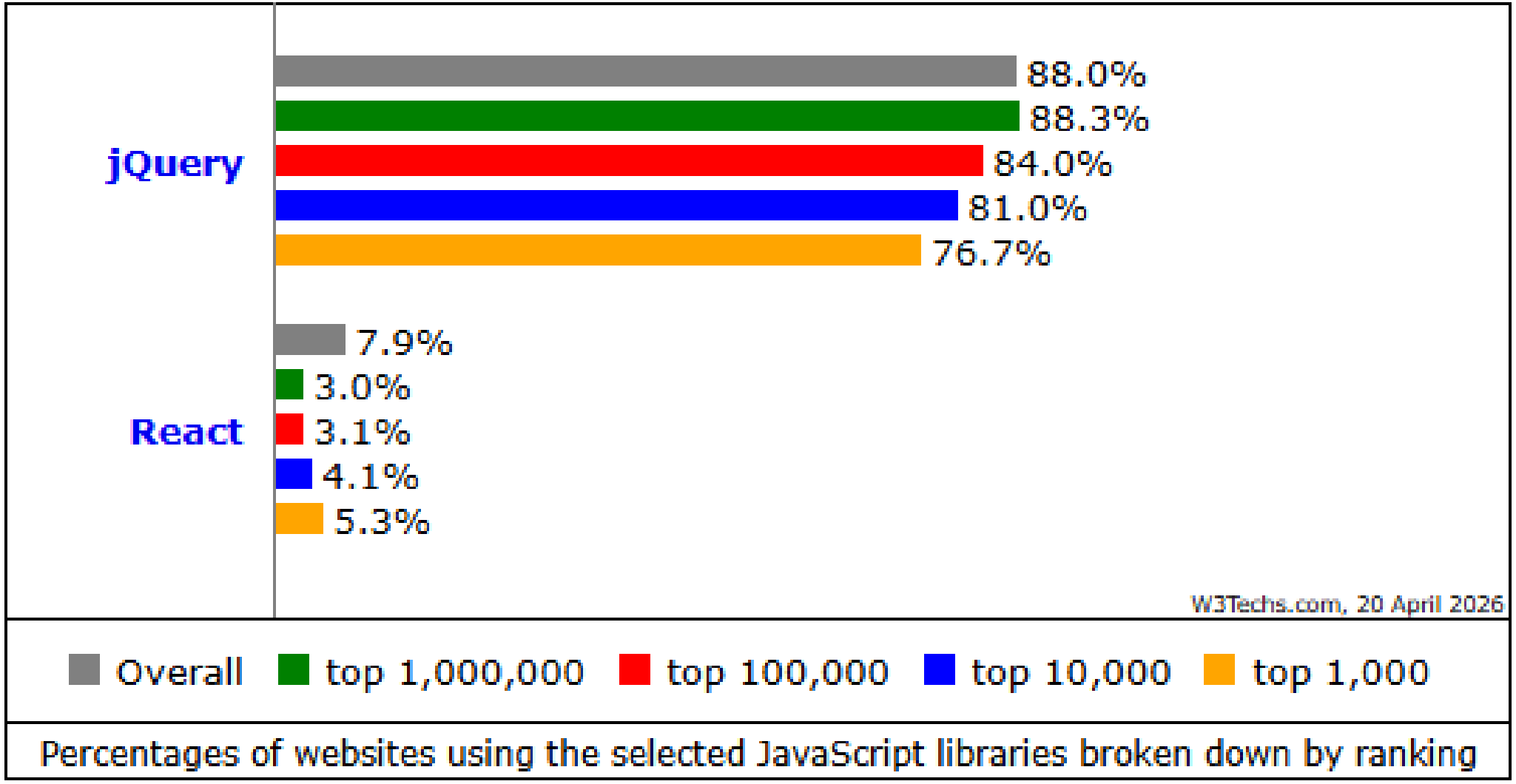
jQuery 4.0 adds support for Trusted Types, ensuring that HTML wrapped in TrustedHTML can be used as input to jQuery manipulation methods in a way that doesn't violate the Content Security Policy directive

Erfarenheter hittills utav jQuery?

jQuery Usage Statistics



jQuery is used by 88.0% of all the websites whose JavaScript library we know.
jQuery is used by 88.3% of all the websites whose JavaScript library we know and that rank in the top 1,000,000.



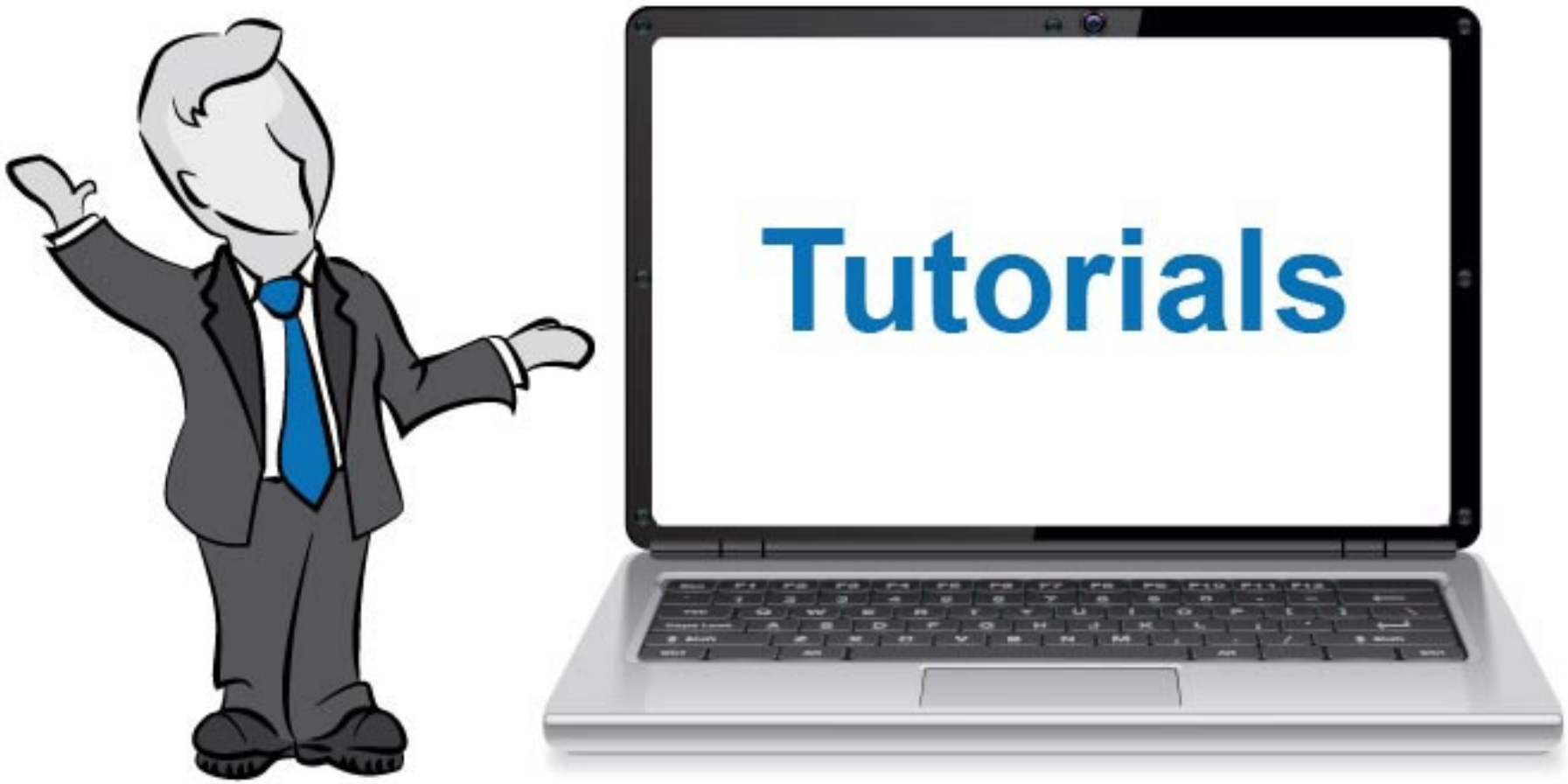
Percentages of websites using the selected JavaScript libraries broken down by ranking

1. jQuery erbjuder ”enklare” kod

2. jQuery erbjuder många bra funktioner

3. jQuery är cross-platform

+ Många plugins



+ Många guider

Exempel på guider

- http://www.w3schools.com/jquery/
- https://www.codecademy.com/learn/learn-jquery
- https://learn.jquery.com/

https://api.jquery.com/

$ = jQuery

$ är en referens till jQuery-objektet

jQuery

- Vi kallar på jQuery när vi vill hitta element på vår webbsida.

jQuery()

Return a collection of matched elements either found in the DOM based on passed argument(s) or created by passing an HTML string.

```
// Letar upp alla länkar i vårt dokument
var links = jQuery("a");
// Letar upp alla element med klassen .blue i vårt dokument
var classBlut = jQuery(".blue");
// Letar upp elementet med id start
var start = jQuery("#start");
```

```
// Letar upp alla länkar i vårt dokument
var links = $("a");
// Letar upp alla element med klassen .blue i vårt dokument
var classBlut = $(".blue");
// Letar upp elementet med id start
var start = $("#start");
```

Jämför med vanligt JavaScript

```
// Letar upp alla länkar i vårt dokument
var links = $("a");
var links = document.querySelectorAll("a");

// Letar upp alla element med klassen .blue i vårt dokument
var classBlue = $(".blue");
var classBlue = document.querySelectorAll(".blue");

// Letar upp elementet med id start
var start = $("#start");
var start = document.querySelector("#start");
```

jQuery-funktioner på element

```
// Letar upp elementet med id start
var start = $("#start");
// Gömmer start
start.hide();
// Visar start
start.show();
// Lägger till klassen "center" på start
start.addClass("start");
// Tar bort start
start.remove();
```

jQuery – Mer än bara hitta element

jQuery har fler väldigt nyttiga funktioner vi kan använda, t.ex.

```
// Skapar ett ajax-anrop
$.ajax();
// Hämtar information från en URL
$.get();
// Loopar igenom en samling element
$.each();
```

jQuery != JavaScript

Vi kan inte använda "vanliga" JavaScript-funktioner på jQuery-objekt, och vice versa.

```
// Inte korrekt
$("#start").setAttribute("class", "center");
document.querySelector("#start").attr("class", "center");

// Korrekt
document.querySelector("#start").setAttribute("class", "center");
$("#start").attr("class", "center");
```

jQuery - Exempel

Snabba exempel – samt att bygga en Todo-lista

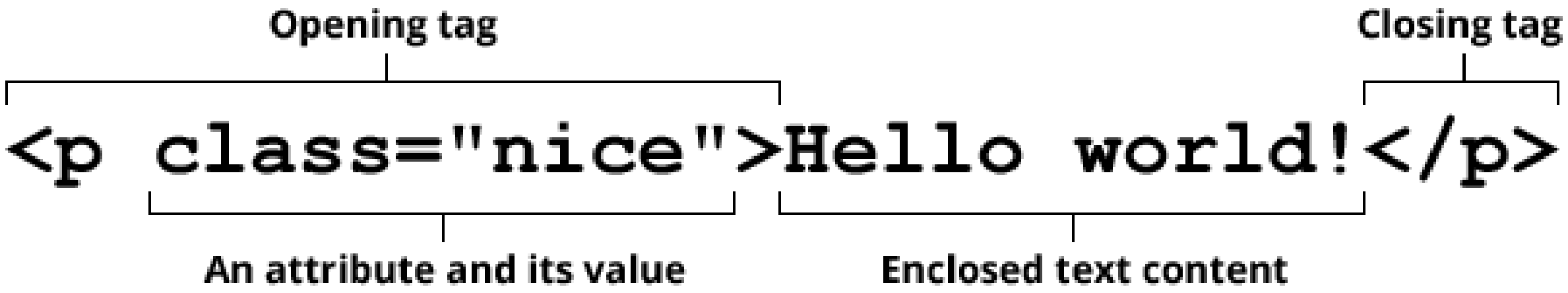
Lär känna din HTML-kod

Vad är syftet med HTML?

Varför har vi element i HTML?

Varför har vi attribut i HTML?

Anatomy of an HTML element



Vad finns det för attribut?

- Många! T.ex.
  - id
  - class
  - title
  - rel
  - src
  - href
  - type
  - … och många fler.
- Beroende vilken typ av element det är - finns det olika attribut som är användbara.

jQuery - Exempel

Att göra flikar!

HTML 5 – Attributet *data-**

- Ibland så räcker inte de attributen som vi har tillgång till. Tänk om jag har behov som dessa inte motsvarar?
- Attributet *data-** ger oss möjlighet att skapa egna attribut där * är ett valfritt namn, t.ex. skulle vi kunna skapa följande attribut för element:

```
<ul>
  <li data-animal-type="bird">Owl</li>
  <li data-animal-type="fish">Salmon</li>
  <li data-animal-type="spider">Tarantula</li>
</ul>
```

jQuery - Hämta attribut

```
// Hämtar värdet för attributet "data-id"
$("#my-element").attr("data-id");
// Sätter värdet för attributet "data-id"
$("#my-element").attr("data-id", "new value");

// Hämtar värdet från attributet "value" (formulärselement)
$("#my-element").val();
// Sätter värdet från attributet "value" (formulärselement)
$("#my-element").val("new value");
```

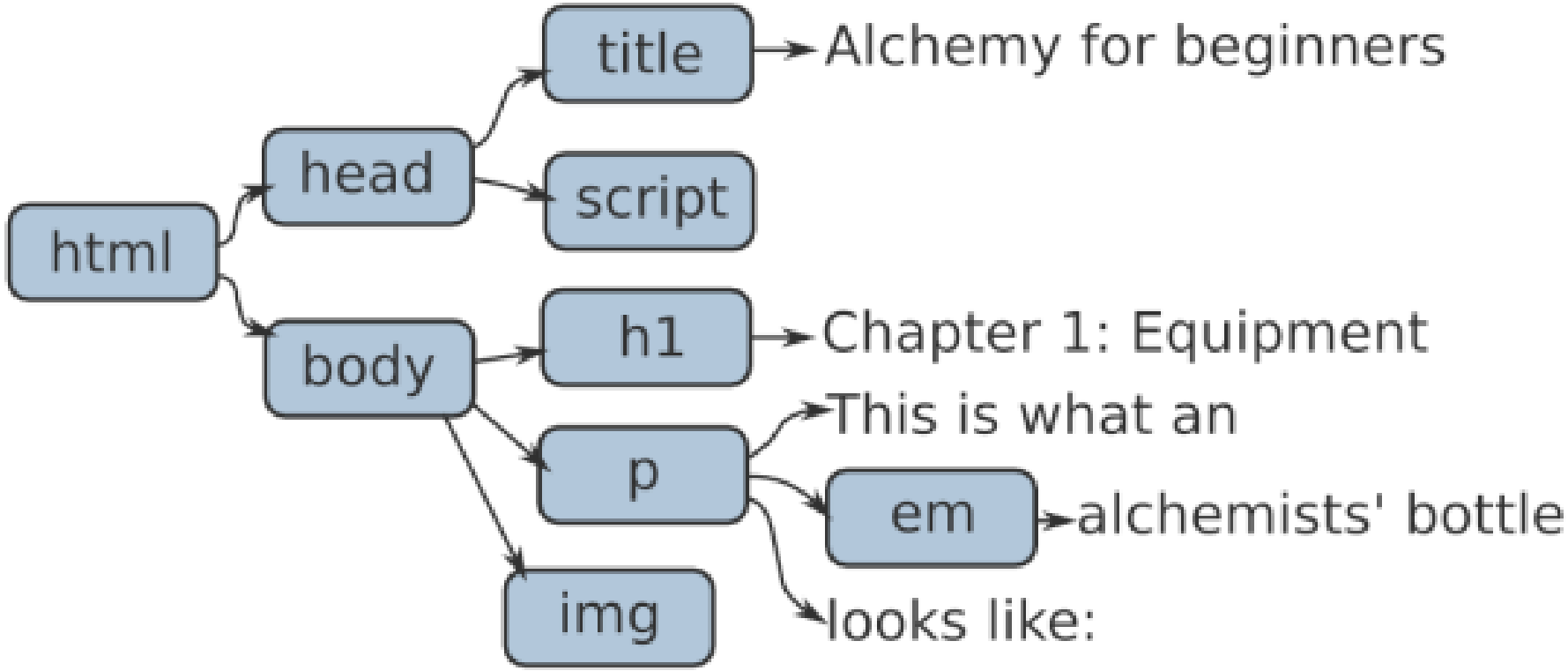
Demo – *data-**

Dags att filtrera data!

DOM

Ännu enklare med jQuery!

DOM


html
head
body
title
script
h1
p
img
Alchemy for beginners
Chapter 1: Equipment
This is what an
em
alchemists' bottle
looks like:

jQuery & DOM

- jQuery har många bra funktioner för att navigera i DOM
  - Som dessutom fungerar cross-browser!
- https://api.jquery.com/category/traversing/tree-traversal/

jQuery & DOM

```
// Nästa element
$("#my-element").next();
// Föregående element
$("#my-element").prev();
// Förälderelement
$("#my-element").parent();
// Elementets barn
$("#my-element").children();
// Letar upp ett element (bland barnen)
$("#my-element").find();
```

jQuery – lägga till innehåll

```
// Lägger till innehåll som första barn till elementet
$("#my-element").prepend("<p>En ny paragraf</p>)");
// Lägger till innehåll som sista barn till elementet
$("#my-element").append("<p>En ny paragraf</p>)");
// Lägger till innehåll före elementet
$("#my-element").before("<p>En ny paragraf</p>)");
// Lägger till innehåll efter elementet
$("#my-element").after("<p>En ny paragraf</p>)");
```

Ajax

Asynchronous JavaScript and XML

Agenda - ajax

- Vad innebär ajax?
- Hur använder man ajax?
- Exempel på ajax
- Att bygga en enkel tjänst med ajax

*" **AJAX** (en. Asynchronous JavaScript and XML) är ett samlingsnamn för flera olika tekniker som kan användas för att bygga applikationer för World Wide Web med bättre interaktivitet än tidigare webbapplikationer."*

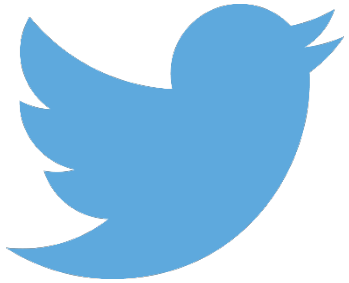
Web 2.0

Aggregators
Folksonomy
Wikis
User Centered
Joy of Use
Blogs
Participation
Six Degrees
Usability
Widgets
Pagerank
XFN
Recommendation
Social Software
FOAF
Browser
Simplicity
Sharing
Collaboration
Perpetual Beta
AJAX
Videocasting
Podcasting
Design
Audio
IM
Video
Convergence
CSS
Pay Per Click
UMTS
Mobility
Atom
XHTML
SVG
Ruby on Rails
VC
Trust
Affiliation
OpenAPIs
RSS
Semantic
Web Standards
SEO
Economy
OpenID
Remixability
REST
Standardization
The Long Tail
DataDriven
Accessibility
XML
Microformats
Syndication
Modularity
SOAP

AJAX

Asynchronous Javascript And XML

Google



YouTube

Exempel: Google

https://www.google.se

”…för flera olika tekniker”

- XMLHttpRequest-objektet, som tillåter JavaScript på en webbsida att göra anrop till en webbserver utan att sidan laddas om.
  - Detta kan var både inom den egna webbplatsen, eller till externa webbplatser
- DOM (Document Object Model) vilket tillåter JavaScript att skriva om innehåll på den aktuella sidan.
- HTML och CSS vilket beskriver utseende för, och märker, innehållet på sidan.
- XML, JSON, m.m. som används för att formatera data som transporteras mellan klient och server. Vanlig text kan också användas som format.

Problem: hämta/skicka information utanför vår html-sida

Lösning: Ajax (oavsett om det är information från vår egen webbplats, eller från annan webbplats)

”Att interaktivt ändra innehållet på vår webbplats”

Från olika data-källor

Exempelområden

- Autocomplete (http://google.com)
  - Vid sökningar av diverse information
- Flerstegsformulär (http://demo.tutorialzine.com/2011/11/chained-ajax-selects-jquery/)
- "Realtidssystem"
  - T.ex. chattsystem, auto-uppdateringar, etc.
- Direktvalidering av formulär
- Autosparning av information
- Röstningssystem
- Etc.

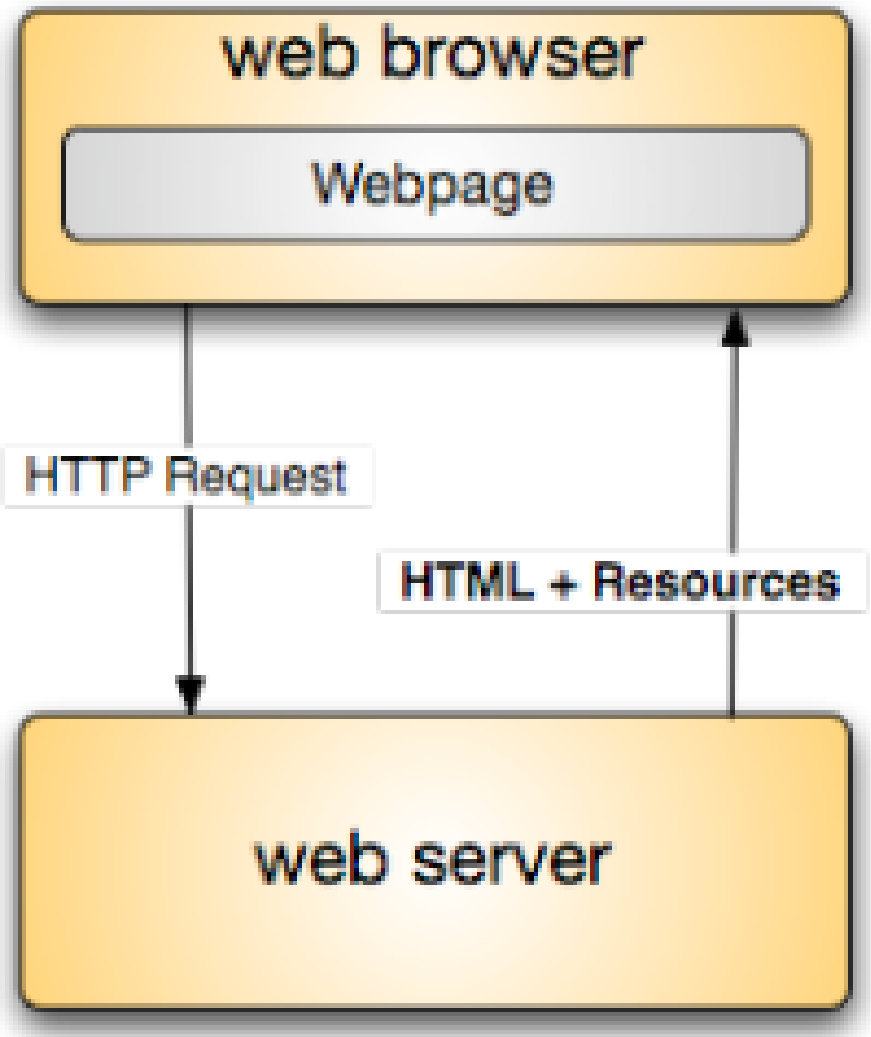
Fördelar med Ajax

- Snabbar upp användandet av webbplatser
  - Ladda bara om den information vi behöver – inte allt
- Färre siduppdateringar på webbplatsen
- Reducerar nätverkstrafik
- Minskar serverbelastning
- Ger ett mer interaktivt GUI

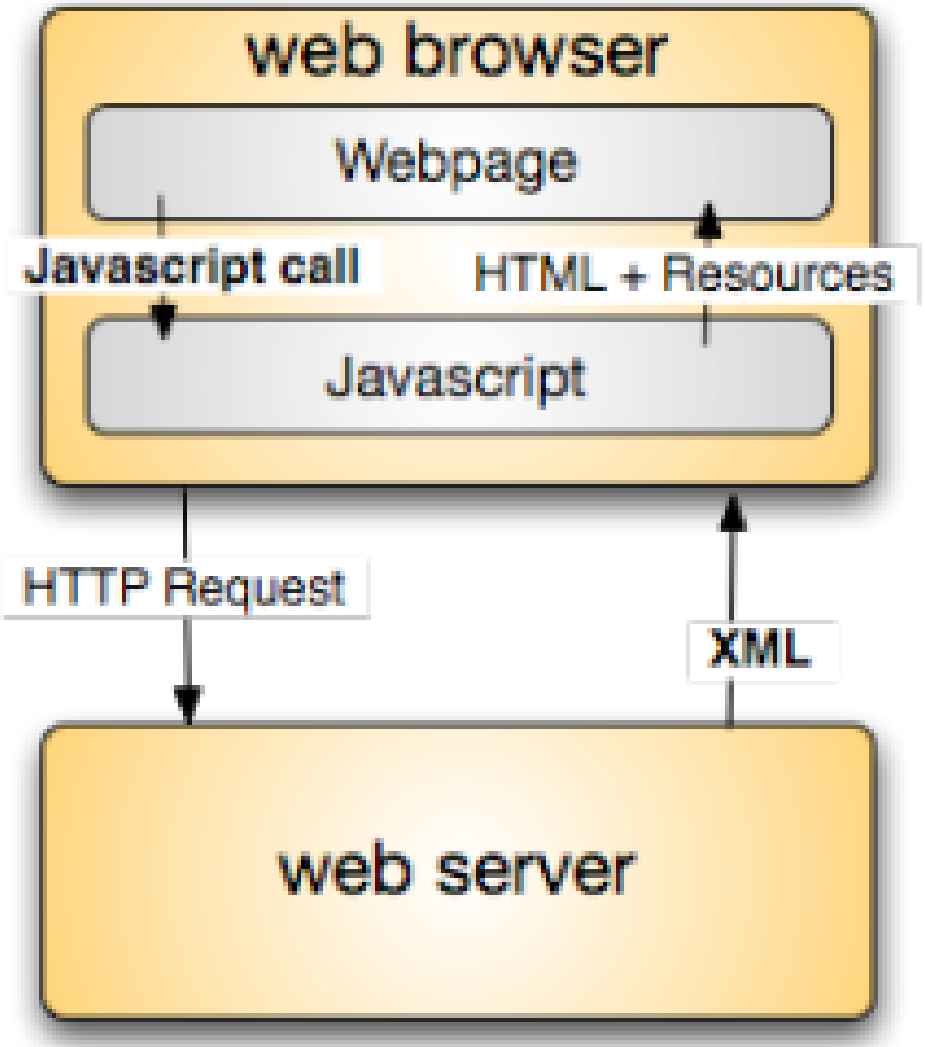
Nackdelar med Ajax

- Bokmärken slutar (ev.) fungera
- Sökmotorer indexerar (ev.) inte innehållet som laddas genom ajax
- ”Bakåtknappen” slutar att fungera
- JavaScript kan enkelt inaktiveras från klientsidan

Traditional web model



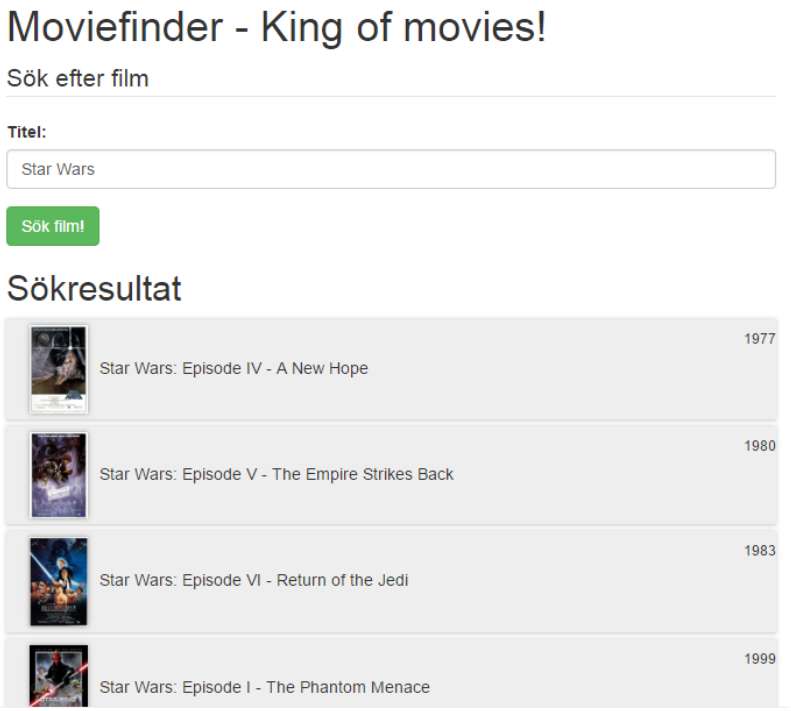
AJAX web model



moviefinder.com/index.html

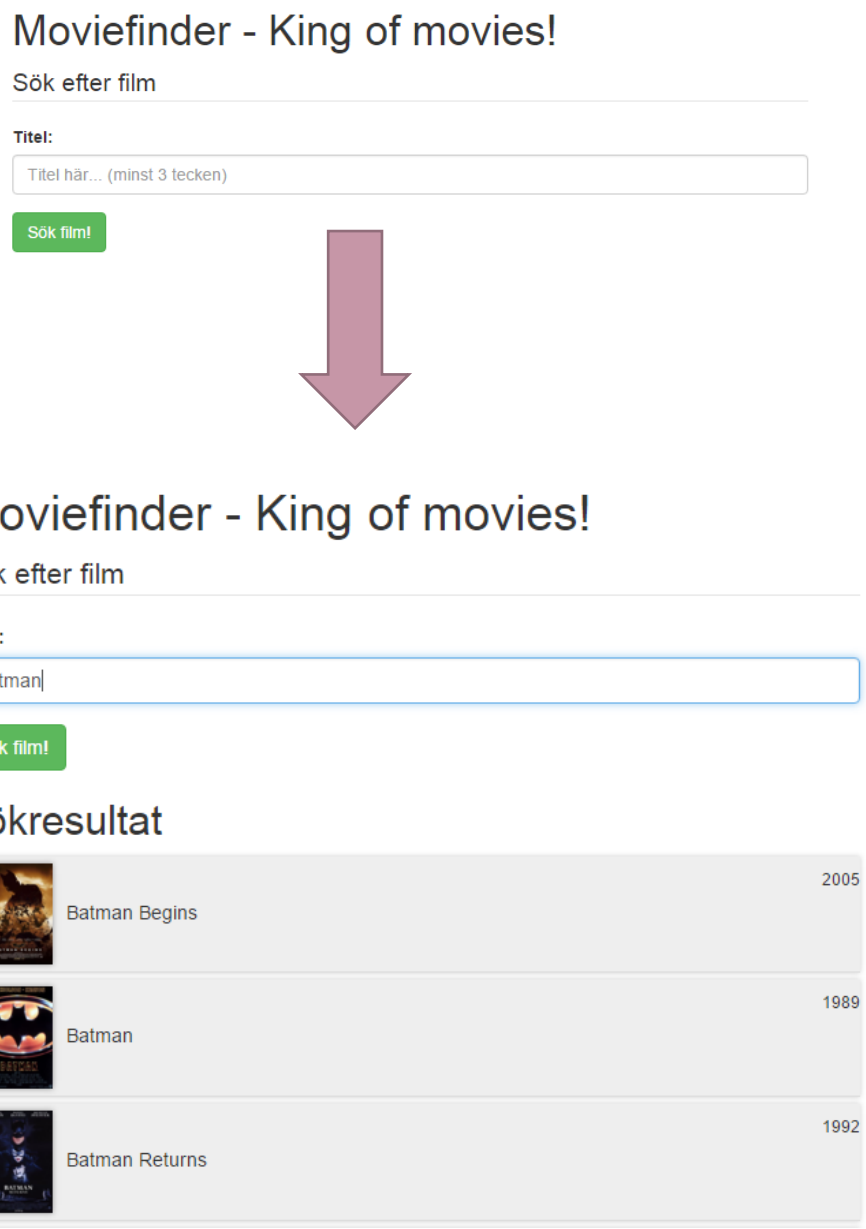
Sidomladdning

moviefinder.com/result.html



- Klassisk webbsida

moviefinder.com/index.html



- Interaktiv webbsida m.h.a. Ajax

Category: Shorthand Methods

These methods perform the more common types of Ajax requests in less code.

jQuery.get()

Load data from the server using a HTTP GET request.

jQuery.getJSON()

Load JSON-encoded data from the server using a GET HTTP request.

jQuery.getScript()

Load a JavaScript file from the server using a GET HTTP request, then execute it.

jQuery.post()

Load data from the server using a HTTP POST request.

.load()

Load data from the server and place the returned HTML into the matched element.

- jQuerys Ajax-metoder

Category: Low-Level Interface

These methods can be used to make arbitrary Ajax requests.

jQuery.ajax()

Perform an asynchronous HTTP (Ajax) request.

jQuery.ajaxPrefilter()

Handle custom Ajax options or modify existing options before each request is sent and before they are processed by $.ajax().

jQuery.ajaxSetup()

Set default values for future Ajax requests. Its use is not recommended.

jQuery.ajaxTransport()

Creates an object that handles the actual transmission of Ajax data.

- jQuerys Ajax-metoder

Exempel på ett ajax-anrop

”data” är ett objekt med svaret som vi får från vårt anrop

”data” är ett objekt med ev. fel som förekommit

```
1
2
3  $.ajax({
4      url: "/"
5  }).done(function(data){
6      console.log("Ajax-anrop klart, lyckades!");
7  }).fail(function(data){
8      console.log("Ajax-anrop klart, misslyckades!");
9  });
10
11
12
```

Asynkront

Bra att veta!

Körning av ajax-anropet

```
1   console.log("Körning 1");
2
3   $.ajax({
4       url: "/"
5   }).done(function(data){
6       console.log("Ajax-anrop klart, lyckades!");
7   }).fail(function(data){
8       console.log("Ajax-anrop klart, misslyckades!");
9   });
10
11  console.log("Körning 2");
12
```

Körning av ajax-anropet

```
1   console.log("Körning 1");
2
3   $.ajax({
4       url: "/"
5   }).done(function(data){
6       console.log("Ajax-anrop klart, lyckades!");
7   }).fail(function(data){
8       console.log("Ajax-anrop klart, misslyckades!");
9   });
10
11  console.log("Körning 2");
12
```

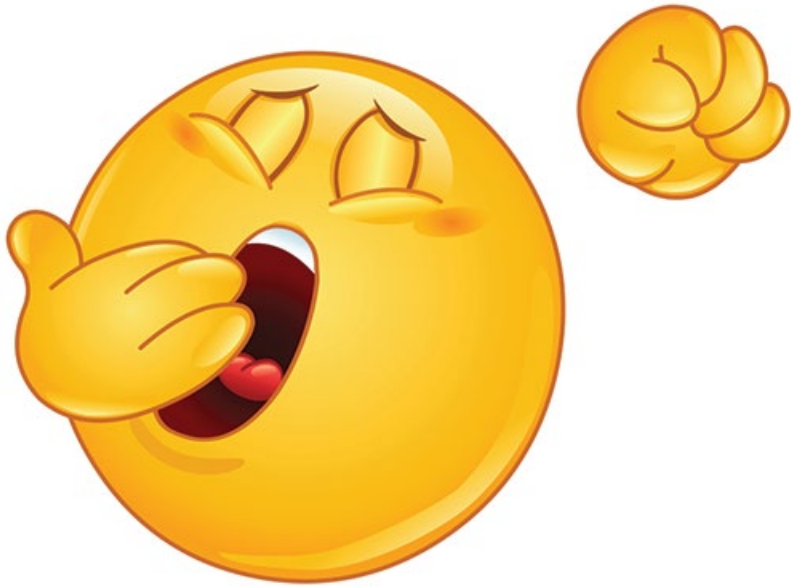
Elements Network Sources Timeline Profiles Resources Console » 1

<top frame> Preserve log

```
Körning 1                                   ajax-temp.js:1
Körning 2                                   ajax-temp.js:10
Ajax-anrop klart, lyckades!                 ajax-temp.js:5
```

Varför asynkront?

- Vi vill inte behöva vänta på slöa servrar! (eller andra saker som kan hindra får sidan från att vara snabb)



Callback-funktioner

- Istället för att göra saker i den ordning som det står (synkront), vill vi istället göra saker när vårt ajax-anrop får ett svar (asynkront).

```
1   console.log("Körning 1");
2
3   $.ajax({
4       url: "/"
5   }).done(function(data){
6       console.log("Ajax-anrop klart, lyckades!");
7   }).fail(function(data){
8       console.log("Ajax-anrop klart, misslyckades!");
9   });
10
11  console.log("Körning 2");
12
```

Utan anonyma funktioner

```
console.log("Körning 1");

$.ajax({
    url: "/"
}).done(onSuccess).fail(onFail);

function onSuccess(data){
    console.log("Ajax-anrop klart, lyckades!");
}

function onFail(data){
    console.log("Ajax-anrop klart, misslyckades!");
}

console.log("Körning 2");
```

Ett standard ajax-anrop

- Med jQ

```
console.log("Körning 1");
$.ajax({
    url: "/", // Adressen för anropet
    type: "GET", // Vilken HTTP-metod vi ska använda
    dataType: "JSON", // Hur vi ska tolka vårt svar
    data: { // Eventuella parametrar vi vill skicka med
        arg1: "value1",
        arg2: "value2",
        arg2: "value3"
    }
}).done(function(data){
    console.log("Ajax-anrop klart, lyckades!");
}).fail(function(data){
    console.log("Ajax-anrop klart, misslyckades!");
});

console.log("Körning 2");
```

API – Ett trevligt sätt att hämta data på.

- Vi vill använda redan färdig data i våra applikationer, t.ex.
- Musik (t.ex. https://developer.spotify.com/)
- Film (t.ex. https://www.omdbapi.com/)
- ”Pappaskämt” (https://icanhazdadjoke.com/)
- … och massvis andra data källor.
- Vi kan även skapa egna data-källor vilket vi ska börja med! =)

Exempel: Raggningsrepliker

Raggningshjälpen!

Ge mig ett tips!

Raggningshjälpen!

Ge mig ett tips!

Tjena kexet, sitter du här och smular?

Förutsättningar

- Vi har en sida som ger oss slumpvalda raggningsrepliker i ren text
  - http://localhost:8080/dating-lines.php
- Vi vill nu varje gång vi klickar på knappen ”Ge mig ett tips”, genom JavaScript (och jQuery) ladda in denna sidas innehåll – och visa upp det

Raggningshjälpen!

Ge mig ett tips!

Tjena kexet, sitter du här och smular?

Låt oss testa!

Använda externa API

https://api.chucknorris.io/

chucknorris.io is a free JSON API for hand curated Chuck Norris facts. Read more

Subscribe for new Chuck Facts

Enter your email

Subscribe

Usage

Retrieve a random chuck joke in JSON format.

```
GET https://api.chucknorris.io/jokes/random
```

Example response:


```
{
"icon_url" : "https://assets.chucknorris.host/img/avatar/chuck-norris.png",
"id" : "Sfx6H9w-Szukf7o_SjEvUg",
"url" : "",
"value" : "Chuck Norris has no enemies, he only has victims"
}
```


Get me a new one ... (press "r" to refresh)

Retrieve a random chuck norris joke from a given category.

Subscribe for new Chuck Facts

Enter your email

Subscribe